Supplementary Material

Experimental Protocol for the First Grover–Rudolph Campaign on the SpinQ Triangulum Platform

Antonio Falcó, Daniela Falcó-Pomares, Juan-Carlos Latorre, *et al.*

Companion to: "Practical Grover–Rudolph state preparation on a 3-qubit NMR quantum computer: from rigorous ancilla-free compilation to staged experimental diagnostics"

Version 1.0 — March 31, 2026

Abstract

This document provides the complete and self-contained experimental protocol underlying the first Grover–Rudolph campaign reported in the companion paper. It is intended to allow an independent research group to reproduce the full experimental workflow—from target distribution specification and circuit compilation through hardware execution, data acquisition, and metric computation—using either the SpinQ Triangulum NMR platform or another device with a compatible native gate set. The protocol is organized into six sequential stages: (§1) prerequisites and hardware setup; (§2) target distribution and Grover–Rudolph angle computation; (§3) circuit compilation and the two ladder variants; (§4) hardware execution and run logging; (§5) data normalization and readout calibration; (§6) metric computation and artifact generation. All numerical constants, gate angles, and software dependencies are given explicitly so that no additional derivation is required.

Contents

1 Prerequisites and hardware setup **1**
- 1.1 Hardware platform . . . 1
- 1.2 Software dependencies . . . 1
- 1.3 Qubit register conventions . . . 2

2 Target distribution and Grover–Rudolph angles **2**
- 2.1 Benchmark target distribution . . . 2
- 2.2 Dyadic probability tree and rotation angles . . . 2
- 2.3 Walsh–Hadamard transform for the uniformly controlled block . . . 3

3 Circuit compilation and ladder variants **3**
- 3.1 Diagnostic stages . . . 3
- 3.2 Compiled circuit for stage $L0$ . . . 4
- 3.3 Compiled circuit for stage $L01$ . . . 4
- 3.4 Compiled circuits for stage FULL: ladders A and B . . . 4

4 Hardware execution and run logging **4**
- 4.1 Run structure . . . 4
- 4.2 Execution order . . . 5
- 4.3 Per-run log record . . . 5
- 4.4 Failed runs . . . 6

5 Data normalization and readout calibration **6**
- 5.1 Probability normalization . . . 6
- 5.2 Per-qubit marginals . . . 6
- 5.3 Readout calibration: confusion matrix . . . 6
- 5.4 Readout error budget . . . 7

6 Metric computation and artifact generation **7**
6.1 Run-level metrics 7
6.2 Output data artifacts 7
6.3 Stage-wise aggregation and statistical comparison 8
6.4 Expected numerical results 8

1 Prerequisites and hardware setup

1.1 Hardware platform

The experiments were carried out on the **SpinQ Triangulum**, a commercial 3-qubit desktop NMR quantum computer [3]. The device provides three qubits (q_0, q_1, q_2) and exposes a native gate dictionary consisting of single-qubit Y-rotations $R_y(\theta)$, bit-flip gates X, and two-qubit entangling gates CNOT. All higher-level operations must be decomposed into this set prior to execution.

Required hardware: SpinQ Triangulum (or equivalent 3-qubit device with native gate set $\{R_y(\cdot), X, \mathrm{CNOT}\}$).

Firmware/driver version: Use the version of the SpinQuasar software package recommended by SpinQ Technology at the time of execution. Record the version number in the run log.

1.2 Software dependencies

The processing pipeline is implemented in Python. The following packages are required:

- `numpy` ≥ 1.24 — numerical computations and array operations.
- `scipy` ≥ 1.10 — statistical tests (Mann–Whitney U).
- `pandas` ≥ 2.0 — tabular data handling and CSV export.
- `jsonlines` ≥ 3.1 — JSONL log serialization.
- SpinQ SDK (version as shipped with the Triangulum unit) — circuit submission and result retrieval.

The complete implementation is available at:

https://github.com/afalco/grover-rudolph-practical-implementation

1.3 Qubit register conventions

Throughout this protocol, the qubit register is ordered as (q_0, q_1, q_2) with q_0 as the most significant bit. Computational basis states are indexed according to

$$|b_0 b_1 b_2\rangle \leftrightarrow k = 4b_0 + 2b_1 + b_2, \quad b_i \in \{0, 1\},$$

so the ordered basis is $\{|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle\}$, corresponding to indices $k = 0, 1, \ldots, 7$.

All qubits are initialized to $|000\rangle$ by the hardware reset procedure before each run. Verify this initialization by checking that a bare measurement (no gates) returns $|000\rangle$ with probability ≥ 0.98. If the bare-state fidelity falls below this threshold, allow the device additional thermal equilibration time before proceeding.

2 Target distribution and Grover–Rudolph angles

2.1 Benchmark target distribution

The target probability vector for the campaign is

$$p = (0.05,\ 0.10,\ 0.15,\ 0.20,\ 0.20,\ 0.15,\ 0.10,\ 0.05), \tag{1}$$

indexed as $p_k = p(|k\rangle)$ for $k = 0, \ldots, 7$. This choice is symmetric around the centre of the register, non-uniform, and sufficiently structured to reveal distributional distortions in the measured output. Verify that $\sum_{k=0}^{7} p_k = 1$ before proceeding.

2.2 Dyadic probability tree and rotation angles

The Grover–Rudolph construction organizes the target law into a dyadic tree of conditional masses [1, 2]. Each internal node at depth d stores the cumulative probability of its subtree; each edge is associated with a conditional R_y rotation angle. For the target (1), the tree and its angles are:

Level 0 (root split, acting on q_0):

$$\theta_0 = 2\arccos\left(\sqrt{\tfrac{p_0+p_1+p_2+p_3}{1}}\right) = 2\arccos\left(\sqrt{0.50}\right) = 90.00^\circ. \tag{2}$$

Level 1 (conditional on q_0, acting on q_1):

$$\theta_{1|0} = 2\arccos\left(\sqrt{\frac{p_0+p_1}{p_0+p_1+p_2+p_3}}\right) = 2\arccos\left(\sqrt{0.30}\right) \approx 113.58^\circ, \tag{3}$$

$$\theta_{1|1} = 2\arccos\left(\sqrt{\frac{p_4+p_5}{p_4+p_5+p_6+p_7}}\right) = 2\arccos\left(\sqrt{0.70}\right) \approx 66.42^\circ. \tag{4}$$

Level 2 (conditional on q_0q_1, acting on q_2):

$$\theta_{2|00} = 2\arccos\left(\sqrt{\tfrac{p_0}{p_0+p_1}}\right) = 2\arccos\left(\sqrt{\tfrac{1}{3}}\right) \approx 109.47^\circ, \tag{5}$$

$$\theta_{2|01} = 2\arccos\left(\sqrt{\tfrac{p_2}{p_2+p_3}}\right) = 2\arccos\left(\sqrt{0.4286}\right) \approx 98.21^\circ, \tag{6}$$

$$\theta_{2|10} = 2\arccos\left(\sqrt{\tfrac{p_4}{p_4+p_5}}\right) = 2\arccos\left(\sqrt{0.5714}\right) \approx 81.79^\circ, \tag{7}$$

$$\theta_{2|11} = 2\arccos\left(\sqrt{\tfrac{p_6}{p_6+p_7}}\right) = 2\arccos\left(\sqrt{\tfrac{2}{3}}\right) \approx 70.53^\circ. \tag{8}$$

Verification step. Before compiling circuits, simulate the full Grover–Rudolph construction using the angles above. Confirm that the ideal output distribution matches the target (1) up to floating-point precision (total variation $\leq 10^{-12}$, fidelity $= 1.000$). Any discrepancy at this stage indicates a formula or implementation error and must be resolved before proceeding to hardware.

2.3 Walsh–Hadamard transform for the uniformly controlled block

At the FULL stage, the four conditional angles $\theta_{2|00}$, $\theta_{2|01}$, $\theta_{2|10}$, $\theta_{2|11}$ must be decomposed into the ancilla-free ladder representation $\{R_y(\cdot), X, \mathrm{CNOT}\}$ using the Walsh–Hadamard-type

angle transform of [2]. Denote the half-angles $\alpha_j = \theta_{2|j}/2$ for $j \in \{00, 01, 10, 11\}$. The Walsh–Hadamard transform gives the ladder rotation angles:

$$\begin{pmatrix} \phi_0 \\ \phi_1 \\ \phi_2 \\ \phi_3 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{10} \\ \alpha_{11} \end{pmatrix}. \tag{9}$$

The resulting values (in degrees) are: $\phi_0 \approx 90.00°$, $\phi_1 \approx 0.03°$ (i.e., $R_y(0.06°)$ for the full angle), $\phi_2 \approx 359.97°$ (i.e., $R_y(719.94°)$), $\phi_3 \approx 6.92°$ (i.e., $R_y(13.84°)$). The assignment to ladder positions differs between ladders A and B (see Section 3).

3 Circuit compilation and ladder variants

3.1 Diagnostic stages

The campaign uses three nested circuit stages that correspond to successive levels of the Grover–Rudolph tree:

- **Stage $L0$:** Only the root rotation on q_0 is applied. The target state has q_1, q_2 nominally in $|0\rangle$, so the ideal output concentrates mass on $|000\rangle$ and $|100\rangle$ only. This stage serves primarily as a calibration diagnostic for readout assignment errors on q_1 and q_2.
- **Stage $L01$:** The root rotation on q_0 and the conditional rotations on q_1 are applied; q_2 remains idle. The ideal output concentrates mass on $|000\rangle$, $|010\rangle$, $|100\rangle$, and $|110\rangle$.
- **Stage FULL:** The complete Grover–Rudolph circuit including the uniformly controlled R_y block on q_2. The ideal output reproduces the target (1) exactly.

3.2 Compiled circuit for stage $L0$

The $L0$ circuit consists of a single $R_y(90°)$ on q_0 followed by two identity R_y pulses on q_0 to match the platform's fixed-depth execution model:

$$\text{Gate sequence on } q_0: \quad R_y(90°),\ R_y(0.06°),\ R_y(719.94°). \qquad [3 \text{ gates}, 3\,R_y]$$

Qubits q_1 and q_2 receive no active gates.

3.3 Compiled circuit for stage $L01$

The $L01$ circuit implements the two-level Grover–Rudolph tree on (q_0, q_1) using the uniformly controlled R_y on q_1 (conditioned on q_0) via a Gray-code 1-control ladder:

$$\text{Gate counts: } 13 \text{ gates } (7\,R_y,\ 4\,\text{CNOT},\ 2\,X).$$

The explicit gate sequence is given in the companion paper (Figure 2b) and in the repository. Qubit q_2 receives no active gates.

3.4 Compiled circuits for stage FULL: ladders A and B

Both FULL circuits implement the complete 3-level tree. They are logically equivalent and share the same gate count:

$$\text{Gate counts: } 21 \text{ gates } (11\,R_y,\ 8\,\text{CNOT},\ 2\,X).$$

They differ *only* in the ordering of control activations in the two-control uniformly controlled R_y block on q_2:

- **Ladder A.** UCR_y control sequence on q_2: (q_1, q_0, q_0, q_0). Sub-rotation order: $R_y(90°)$, $\mathrm{CNOT}(q_1)$, $R_y(5.63°)$, $\mathrm{CNOT}(q_0)$, $R_y(360°)$, $\mathrm{CNOT}(q_0)$, $R_y(13.84°)$, $\mathrm{CNOT}(q_0)$.
- **Ladder B.** UCR_y control sequence on q_2: (q_1, q_1, q_0, q_0). Sub-rotation order: $R_y(90°)$, $\mathrm{CNOT}(q_1)$, $R_y(13.84°)$, $\mathrm{CNOT}(q_1)$, $R_y(360°)$, $\mathrm{CNOT}(q_0)$, $R_y(5.63°)$, $\mathrm{CNOT}(q_0)$.

The rationale for testing both variants is that, while they are unitarily identical, their pulse schedules differ on NMR hardware and may therefore produce different experimental fidelities.

Pre-flight verification. Before submitting any circuit to hardware:

1. Run the ideal simulator for all four circuits ($L0$, $L01$, FULL-A, FULL-B).
2. Confirm that the simulated output for FULL-A and FULL-B both match the target (1) with TV $\leq 10^{-12}$.
3. Confirm that $L0$ and $L01$ match their respective truncated target distributions exactly.
4. Record the simulator outputs as the reference vectors for subsequent metric computation.

4 Hardware execution and run logging

4.1 Run structure

The campaign is organized as follows:

- **Total runs:** 150 hardware executions.
- **Per stage:** 50 runs ($L0$: 50, $L01$: 50, FULL: 50).
- **Per ladder (FULL stage only):** 25 runs for ladder A and 25 runs for ladder B.
- **Shots per run:** as specified by the platform's default single-run acquisition mode (approximately 4096 shots for the SpinQ Triangulum); record the exact number in the log.

4.2 Execution order

To minimize systematic drift effects, runs should be interleaved across stages and ladder variants. The recommended block structure is:

1. Execute one block of 25 runs for $L0$.
2. Execute one block of 25 runs for $L01$.
3. Execute one block of 25 runs for FULL-A.
4. Execute one block of 25 runs for FULL-B.
5. Repeat the full sequence once to complete 50 runs per stage and 25 runs per ladder.

If the hardware session must be interrupted, note the run index and timestamp. Do not merge data from sessions separated by more than 24 hours without logging a device restart and re-verifying the bare-state fidelity.

4.3 Per-run log record

For each hardware run, record the following fields in JSONL format (one JSON object per line):

Field	Type	Description
`run_name`	string	Unique run identifier (e.g. `L0_A_001`)
`stage`	string	`L0`, `L01`, or `FULL`
`ladder`	string	`A` or `B` (`FULL` only; else `A`)
`repeat`	integer	Run index within the stage–ladder group (1-based)
`platform_run_id`	integer	Internal job ID returned by the platform SDK
`status`	string	`completed` or `failed`
`created`	string	ISO 8601 timestamp of job submission
`end`	string	ISO 8601 timestamp of result retrieval
`duration_s`	float	Wall-clock duration in seconds
`n_qubits`	integer	3
`n_gates`	integer	Total gate count
`n_ry`	integer	Number of R_y gates
`n_x`	integer	Number of X gates
`n_cnot`	integer	Number of CNOT gates
`raw_counts`	object	Dictionary of bitstring $\rightarrow$ shot count
`exp_probs`	object	Normalized experimental probability vector
`sim_probs`	object	Ideal simulator probability vector

A minimal Python snippet for log serialization:

Listing 1: Per-run log serialization.

```python
import json

def log_run(record: dict, logfile: str) -> None:
    """Append one run record to a JSONL file."""
    with open(logfile, "a", encoding="utf-8") as f:
        f.write(json.dumps(record, ensure_ascii=False) + "\n")
```

4.4 Failed runs

Mark any run with `status = "failed"` and exclude it from all subsequent analyses. Do *not* re-run failed jobs within the same session without first inspecting the platform logs. If more than 10% of runs within a single block fail, suspend the campaign and contact the hardware operator.

5 Data normalization and readout calibration

5.1 Probability normalization

After retrieval, convert raw shot counts to normalized probability vectors:

$$q_k = \frac{c_k}{\sum_{j=0}^{7} c_j}, \quad k = 0, \ldots, 7, \tag{10}$$

where c_k is the count for bitstring k. If the total count deviates by more than 1% from the nominal shot number, log a warning but do not discard the run.

5.2 Per-qubit marginals

Compute single-qubit marginal probabilities for each qubit q_j:

$$P(q_j = b) = \sum_{\substack{k=0 \\ b_j(k)=b}}^{7} q_k, \quad b \in \{0, 1\}, \tag{11}$$

where $b_j(k)$ denotes the j-th bit of the binary representation of k. Store $P(q_j = 0)$ and $P(q_j = 1)$ for each run.

5.3 Readout calibration: confusion matrix

The $L0$ stage provides a direct readout calibration diagnostic. At $L0$, qubits q_1 and q_2 are nominally idle and should measure $|0\rangle$ with probability 1. Any deficit is a direct measure of the $0 \to 1$ assignment error:

$$\varepsilon_{0\to1}(Q_j) = 1 - P_{\text{exp}}(q_j = 0)\big|_{L0}, \quad j \in \{1, 2\}. \tag{12}$$

Pool all 50 $L0$ runs to estimate the mean and standard deviation of $\varepsilon_{0\to1}$ for each qubit.

A full 8×8 confusion matrix Λ can optionally be estimated by preparing each of the 8 basis states and recording the output distribution. The (i, j) entry gives the probability of measuring $|j\rangle$ when $|i\rangle$ was prepared:

$$\Lambda_{ij} = P(\text{measure } j \mid \text{prepare } i). \tag{13}$$

Readout-mitigated probability vectors are obtained by solving the linear system $\Lambda\hat{q} = q_{\text{exp}}$ and projecting $\hat{q}$ onto the probability simplex.

> **Note on confusion matrix estimation.** Calibrating the full 8×8 matrix requires 8 additional circuit types (one per basis state). In the present campaign, calibration was not applied as a post-processing correction to the FULL-stage data; instead, the $L0$ stage was used to extract per-qubit assignment errors only. Future campaigns should consider a separate calibration session with full 8×8 estimation if per-bitstring correction is desired.

5.4 Readout error budget

For each run, compute the readout TV gap

$$\Delta\,\text{TV} = \text{TV}(q_{\text{exp}},\, p) - \text{TV}(q_{\text{exp}},\, q_{\text{sim}}), \tag{14}$$

where p is the target (1) and q_{sim} is the ideal simulator output. This gap isolates the contribution of readout bias to the total discrepancy; gate- and coherence-level errors appear in $\text{TV}(q_{\text{exp}}, q_{\text{sim}})$ only.

6 Metric computation and artifact generation

6.1 Run-level metrics

For each completed run with experimental distribution q and simulator distribution q_{sim}, compute the following metrics against both references:

Total variation distance:

$$\text{TV}(p, q) = \frac{1}{2}\sum_{k=0}^{7} |p_k - q_k|. \tag{15}$$

ℓ^2 distance:

$$\|p - q\|_2 = \left(\sum_{k=0}^{7} (p_k - q_k)^2 \right)^{1/2}. \tag{16}$$

Classical fidelity:

$$\mathrm{Fid}(p, q) = \left(\sum_{k=0}^{7} \sqrt{p_k \, q_k} \right)^2. \tag{17}$$

Compute all six combinations: `tv_vs_sim`, `l2_vs_sim`, `fidelity_vs_sim`, `tv_vs_target`, `l2_vs_target`, `fidelity_vs_target`.

6.2 Output data artifacts

The following four tabular artifacts must be generated and stored:

1. `runs_flat.csv` — one row per run, with all fields from the log record plus the six computed metrics and the per-qubit marginals (12 columns: `sim_q0_p0`, ..., `exp_q2_p1`) and the full probability vectors (16 columns: `sim_p_000`, ..., `exp_p_111`).
2. `summary_by_ladder_stage.csv` — one row per (ladder, stage) group, with means and standard deviations of all metrics and means of probability vectors. The six groups are: $(L0, A)$, $(L0, B)$, $(L01, A)$, $(L01, B)$, $(FULL, A)$, $(FULL, B)$.
3. `circuits_flat.csv` — one row per run (may be shared across runs with the same stage and ladder), with gate counts, gate type sequence, and the raw circuit and simulation JSON payloads.
4. `runs_full.jsonl` — the complete per-run records in JSONL format, preserving all raw fields and metadata for programmatic access.

6.3 Stage-wise aggregation and statistical comparison

Aggregate the run-level metrics by (stage, ladder) group: compute the mean and standard deviation of each metric across the 25 runs in each group, and the mean of each probability vector component.

To test whether ladders A and B differ at the FULL stage, apply the two-sided Mann–Whitney U test independently for each metric (`tv_vs_target`, `l2_vs_target`, `fidelity_vs_target`). Report the U statistic, the two-sided p-value, and the rank-biserial correlation r. The null hypothesis is that the two ladder groups are drawn from the same distribution. Use significance level $\alpha = 0.05$.

Listing 2: Metric computation and Mann–Whitney test.

```python
import numpy as np
from scipy.stats import mannwhitneyu

def tv(p: np.ndarray, q: np.ndarray) -> float:
    return 0.5 * np.sum(np.abs(p - q))

def l2(p: np.ndarray, q: np.ndarray) -> float:
    return np.sqrt(np.sum((p - q) ** 2))

def fidelity(p: np.ndarray, q: np.ndarray) -> float:
    return np.sum(np.sqrt(p * q)) ** 2

def mw_test(x: np.ndarray, y: np.ndarray):
```

```
    U, pval = mannwhitneyu(x, y, alternative="two-sided")
    n1, n2 = len(x), len(y)
    r = 1 - 2 * U / (n1 * n2)       # rank-biserial correlation
    return {"U": U, "p": pval, "r": r}
```

6.4 Expected numerical results

For the campaign reported in the companion paper, the stage-wise aggregated results (means ± standard deviations over 50 runs per stage) are:

Table 2: Expected stage-wise performance metrics. Values reproduced from the companion paper. An independent replication should recover results within the reported standard deviations.

Stage	Runs	TV vs target	ℓ^2 vs target	Fidelity vs target
$L0$	50	0.526 ± 0.040	0.460 ± 0.032	0.652 ± 0.043
$L01$	50	0.348 ± 0.042	0.294 ± 0.033	0.810 ± 0.043
FULL	50	0.233 ± 0.053	0.198 ± 0.042	0.913 ± 0.035

Deviations beyond 2σ from these reference values should be investigated (device recalibration, firmware update, or environmental disturbance) before the results are used for comparison.

Data and code availability

All data artifacts (`runs_flat.csv`, `summary_by_ladder_stage.csv`, `circuits_flat.csv`, `runs_full.jsonl`) and the full Python implementation are publicly available at:

https://github.com/afalco/grover-rudolph-practical-implementation

The repository includes a `README` with step-by-step instructions for reproducing the analysis from the raw JSONL log.

References

[1] Grover, L., Rudolph, T.: Creating superpositions that correspond to efficiently integrable probability distributions. arXiv:quant-ph/0208112 (2002)

[2] Falcó, A., Falcó-Pomares, D., Matthies, H.G.: A rigorous and self-contained proof of the Grover–Rudolph state preparation algorithm. arXiv:2601.17930 (2026). Submitted to AIMS Mathematics

[3] Feng, G., Hou, S.-Y., Zou, H., et al.: SpinQ Triangulum: a commercial three-qubit desktop quantum computer. arXiv:2202.02983 (2022)